FuzzyTutor

Technical Project Report, Fuzzy-System Definition, Evaluation, and User Manual

13 July 2026

Executive summary

FuzzyTutor is a web-based adaptive tutor for programming concepts. It was designed as a joint project for the Fuzzy Logic and Systems course and the User Experience and Artificial Intelligence course. The system combines two complementary inference engines:

1. a first-order Takagi--Sugeno ANFIS implementation that predicts a continuous Knowledge Mastery score; and
2. a Mamdani fuzzy inference system that estimates System Cognitive Friction from observable learning behavior.

The backend synthesizes both outputs to select the difficulty of the next uncompleted task. The user interface can explain the decision through mastery and friction gauges, a focus-state label, a support message, and a human-readable adaptation reason. Short satisfaction surveys are requested at persistent five-task milestones.

The application uses Django and Django REST Framework for the backend, React and Vite for the frontend, SQLite for assignment-scale persistence, and Docker Compose for reproducible execution. The architecture is decoupled: the frontend consumes explicit JSON contracts and does not contain fuzzy rules or answer keys.

No historical learner dataset was available. Therefore, ANFIS training uses a deterministic synthetic bootstrap dataset that represents five plausible behavioral profiles. This is appropriate for demonstrating the full training and inference pipeline at assignment scale, but it is not presented as evidence about a real student population. A fixed 80/20 holdout split is used so evaluation rows are not used to fit model parameters.

Requirements and scope

The implemented system addresses the final joint project definition as follows:

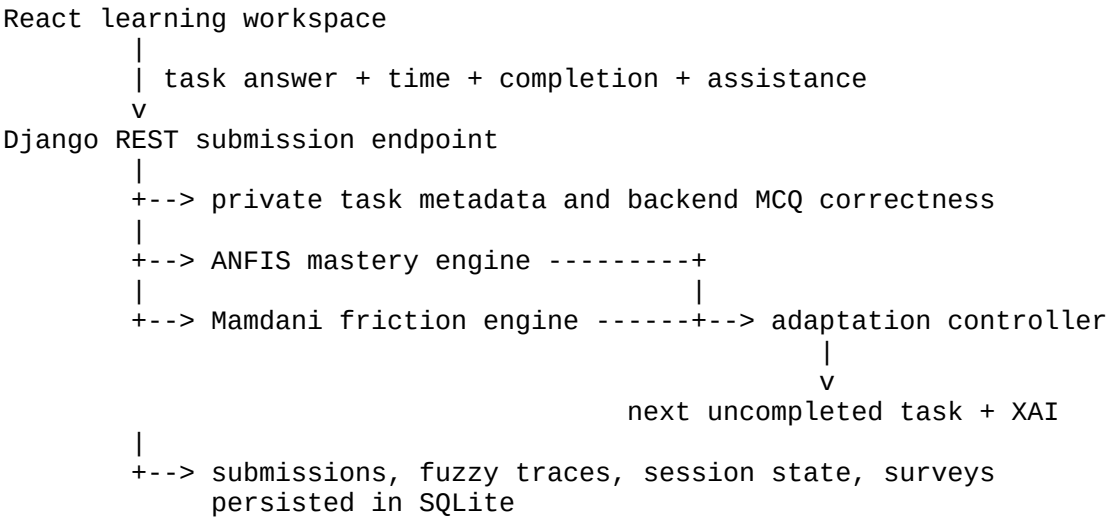
Requirement	Implementation
Personalized fuzzy decisions	Continuous mastery/friction estimates drive task difficulty and support messages
Fuzzy sets and membership functions	Explicit triangular and shoulder functions in both engines
Fuzzy rules	Five ANFIS activation rules and six core

Requirement	Implementation
	Mamdani rules plus a code-workspace rule
Defuzzification	ANFIS normalized weighted TSK output; Mamdani centroid of aggregated output area
ANFIS extension	Trainable first-order consequent parameters with a versioned model artifact
Seven programming modules	Lists, arrays, dictionaries, classes, inheritance, exceptions, and control flow
MCQ and code-writing tasks	Private answer-key MCQs and non-graded code workspace tasks
Dynamic adaptation	Harder, equivalent, or easier uncompleted task chosen from the curriculum
Explainable AI	Engine memberships, active rules, scores, focus state, recommendation, and friendly reason
Feedback loop	Persistent micro-survey every five completed tasks and telemetry export
Decoupled web application	Django REST API and React frontend in separate containers

The backend is intentionally assignment-scale. It uses anonymous session tokens and SQLite rather than production identity, authorization, and distributed infrastructure. Those choices keep the implementation inspectable during assessment and do not affect the fuzzy-system demonstration.

System architecture

The operational data flow is:



The backend is separated into three Django applications:

- `core`: health and shared service concerns;

- `learning`: curriculum, learner sessions, submissions, surveys, telemetry, and adaptation; and
- `fuzzy`: membership functions, ANFIS, Mamdani inference, training, evaluation, and synthesis.

Data model

`LearnerSession` stores the anonymous session token, current module/task, rolling mastery/friction aggregates, completed-task count, and latest recommendation.

`TaskSubmission` stores task metadata at the moment of submission, elapsed and relative response time, assistance count, completion ratio, backend-derived MCQ correctness, and the answer payload. Code-task correctness remains null because code is not automatically graded.

`FuzzyEvaluationLog` stores a reproducible input snapshot, complete engine trace, both crisp outputs, focus state, recommendation, and support message.

`MicroSurveyResponse` stores satisfaction, perceived difficulty, confidence, optional comments, and the exact five-task milestone. A uniqueness constraint prevents duplicate responses for one session milestone.

Learning content and interaction design

The task bank contains three difficulty bands:

Band	Level	Task metric weight	Estimated cognitive load
Foundation	1	35	Low
Intermediate	2	55	Medium
Advanced	3	75	High

Each of the seven modules contains MCQ and/or code-writing tasks. MCQ choices are returned to the browser, but the correct answer remains private on the backend. Code tasks return starter code but not an answer guide.

Code submissions are deliberately not executed or graded. This prevents compiler/test feedback from becoming an additional anxiety signal. Instead, the models use completion ratio, normalized response time, assistance interactions, task challenge, and prior aggregate mastery. A code response is never stored as correct merely because it is non-empty.

Shared fuzzy membership functions

For a value x, the triangular membership function with left point a, peak b, and right point c is:

$\mu_{triangle}(x; a,b,c) = 0$ outside $[a,c]$; $(x-a)/(b-a)$ for $a < x < b$; and $(c-x)/(c-b)$ for $b \le x < c$.

The left-shoulder function is 1 at and below a, decreases linearly between a and b, and is 0 at and above b. The right-shoulder function is its mirror: 0 at and below a, increases linearly, and is 1 at

and above b.

All final crisp scores are clamped to the interval [0,100].

ANFIS cognitive-performance engine

Inputs and output

The cognitive engine predicts Knowledge Mastery on [0,100]. Its evidence is:

- Task Metric Weight on [0,100];
- Historical Grade Average, represented operationally by the session's rolling mastery, on [0,100];
- Completion Ratio on [0,1];
- a correctness/performance signal; and
- task type, used only for a small incomplete-code penalty feature.

For MCQs, correctness is computed against the private backend key. For code tasks, correctness is unknown; a bounded performance signal is derived from completion without claiming that the code is correct.

Premise membership sets

Variable	Linguistic set	Function and parameters
Challenge	Foundation	left shoulder (35, 60)
Challenge	Intermediate	triangle (35, 58, 82)
Challenge	Advanced	right shoulder (65, 85)
History	Low	left shoulder (35, 60)
History	Medium	triangle (45, 68, 86)
History	High	right shoulder (72, 90)
Completion	Low	left shoulder (0.20, 0.65)
Completion	Medium	triangle (0.35, 0.70, 0.98)
Completion	High	right shoulder (0.75, 1.00)
Performance	Weak	left shoulder (35, 65)
Performance	Emerging	triangle (45, 70, 90)
Performance	Strong	right shoulder (75, 95)

Rule layer

Rule activation uses fuzzy AND = minimum and fuzzy OR = maximum.

1. **Secure prior mastery:** high history AND high completion AND strong performance.
2. **Developing mastery:** (medium history AND high completion) OR (high history AND emerging performance).
3. **Productive challenge:** advanced challenge AND high completion AND strong performance.
4. **Fragile progress:** (medium history AND medium completion) OR (emerging

performance AND medium completion).

5. **Knowledge gap:** (low history AND weak performance) OR (low completion AND weak performance).

Each rule has a first-order TSK consequent:

$f_i = p_i H + q_i C + r_i P + s_i W + t_i K + b_i$

where H is history, C is completion percentage, P is the performance signal, W is task weight, K is the incomplete-code penalty, and b is a bias. The crisp mastery prediction is the normalized weighted mean:

$\text{Mastery} = \Sigma(w_i f_i) / \Sigma(w_i)$

where w_i is the activation strength of rule i. This corresponds to the core ANFIS layers: premise fuzzification, rule firing, normalization, consequent calculation, and output aggregation. The premise definitions are fixed for transparency; the consequent parameters are trained.

Synthetic training data

The deterministic generator creates 180 records across five profiles: strong, struggling, improving, overchallenged, and fast-incorrect. Each row includes task weight, history, response-time ratio, assistance, completion, task type, correctness state, confidence, perceived difficulty, and a target mastery label.

The synthetic target is a declared weighted teaching heuristic, not hidden ground truth:

Target = 0.42(correctness) + 0.24(completion) + 0.14(speed) + 0.12(confidence) + 0.08(history) + difficulty adjustment.

The seed is 42. A deterministic split holds out 20% of records before gradient fitting. The versioned model was trained for 650 epochs at learning rate 0.00002.

Holdout evaluation

Metric	Default consequent baseline	Trained model
Holdout records	36	36
MAE	13.052	4.334
RMSE	14.317	5.840
R^2	0.532	0.922

The RMSE improvement is 8.477 points on held-out synthetic samples. These results demonstrate that the implemented training pipeline learns its declared synthetic relationship. They do not establish effectiveness for real learners; real-data calibration is future work.

Mamdani cognitive-friction engine

Inputs and premise sets

The behavioral engine predicts System Cognitive Friction on [0,100]. It uses response time relative to the task baseline, assistance interactions, completion ratio, and whether the workspace is a code task.

Variable	Set	Function and parameters
Relative response time	Low	left shoulder (0.45, 0.95)
Relative response time	Normal	triangle (0.65, 1.00, 1.45)
Relative response time	High	right shoulder (1.15, 2.10)
Assistance	Low	left shoulder (0.0, 1.5)
Assistance	Medium	triangle (0.5, 2.0, 3.5)
Assistance	High	right shoulder (2.5, 5.0)
Completion	Incomplete	left shoulder (0.15, 0.55)
Completion	Partial	triangle (0.25, 0.60, 0.95)
Completion	Complete	right shoulder (0.70, 1.00)

The output linguistic sets are:

Friction set	Function and parameters
Low	left shoulder (15, 35)
Moderate	triangle (20, 42, 64)
High	triangle (48, 72, 90)
Severe	right shoulder (72, 92)

Mamdani rules

1. Low time pressure AND low assistance AND complete → low friction.
2. Normal time AND complete → low friction.
3. (Normal time AND medium assistance) OR (high time AND complete) → moderate friction.
4. Partial completion AND (normal time OR medium assistance) → moderate friction.
5. High time AND (medium assistance OR high assistance) → high friction.
6. (Incomplete AND high time) OR (incomplete AND high assistance) → severe friction.
7. A code workspace activates moderate friction at strength 0.35 to represent its additional interaction load without treating it as failure.

Inference and centroid defuzzification

Antecedents use minimum for AND and maximum for OR. Each active rule clips its consequent membership function at the rule strength. Consequents are aggregated pointwise with maximum over a [0,100] universe sampled at 0.25-point resolution.

The crisp result is the centroid of the aggregated output area:

Friction = Σ(x μ_aggregated(x)) / Σ(μ_aggregated(x)).

The API trace returns the input memberships, active rule strengths, output-set definitions, aggregated area, and the explicit method name `centroid_of_aggregated_output_area`.

Synthesis and adaptation controller

The controller maps both crisp scores to a calm user-facing state:

- **Focused & Steady:** friction < 25 and mastery ≥ 65;
- **Needs Support:** friction < 55 when the steady condition is not met; and
- **Frustrated:** friction ≥ 55.

The next difficulty decision is:

Condition	Recommendation	Action
Mastery ≥ 75 and friction < 35	Increase or hold high tier	Target one level harder, capped at advanced
Mastery < 45 and friction ≥ 55	Reduce difficulty and support	Target one level easier, floored at foundation
Mixed evidence	Hold current tier	Stay near the current level

The selector never repeats a completed task while uncompleted curriculum tasks remain. It first searches the current module near the target difficulty. When the module is complete, it advances through later modules. When all tasks are complete, it sets `curriculumComplete=true` rather than silently cycling old work.

Session aggregates use a smoothing update: 65% prior aggregate plus 35% latest engine output. This prevents one unusually fast, slow, correct, or incomplete response from causing an abrupt learner-profile change.

Explainability and feedback

Every submission response contains:

- Knowledge Mastery and System Cognitive Friction;
- focus state, recommendation, and support message;
- next-task difficulty and a plain-language reason;
- input snapshot; and
- optional detailed memberships and active-rule trace.

The primary interface should show the friendly interpretation rather than raw rule identifiers. A detailed trace can be placed in an expandable assessment/debug section.

At every five completed tasks, the backend exposes `surveyDue=true`. The oldest unanswered milestone remains due across refreshes. A survey records 1--5 satisfaction, perceived difficulty, and confidence scores plus an optional comment. Duplicate responses for the same milestone are rejected. The export endpoint joins these labels to task and model telemetry for later analysis.

Because no live study has yet been conducted, this report makes no fabricated claims about student or educator satisfaction. The implemented feedback mechanism, persistence, and export contract support a future evaluation with both groups. A suitable study would compare perceived difficulty with model friction, examine satisfaction before and after adaptations, and collect educator judgments about recommendation appropriateness.

REST API contract

Method and path	Purpose
`GET /api/health/`	Service status
`GET /api/learning/modules/`	Seven modules and task counts
`GET /api/learning/tasks/`	Public, answer-safe task catalog
`POST /api/learning/sessions/`	Create anonymous learning session
`GET /api/learning/sessions/{token}/`	Restore session and pending survey state
`POST /api/learning/submissions/`	Persist response, run both engines, adapt next task
`POST /api/learning/micro-surveys/`	Save the currently due survey milestone
`GET /api/learning/export/training-data/`	Export joined telemetry for assignment analysis
`POST /api/fuzzy/evaluate/`	Standalone engine demonstration without persistence
`GET /api/docs/`	Typed Swagger/OpenAPI documentation

Malformed or unknown module identifiers, invalid navigation directions, stale/non-current task submissions, client-supplied correctness, duplicate surveys, and unknown tokens receive 4xx responses rather than server errors.

User manual

Starting the system

1. Install Docker with Docker Compose support.
2. From the project directory, run `docker compose up --build`.
3. Open `http://localhost:5173` for the tutor.
4. Open `http://localhost:8000/api/docs/` to inspect or demonstrate the backend contracts.

The frontend and backend run in separate containers. Database migrations run when the backend container starts.

Learner workflow

1. Start or restore a learner session.
2. Select the answer to an MCQ or write a response in the code editor.
3. Submit the task. The code editor does not run automated tests.
4. Read the mastery/friction gauges and the short support message.

5. Continue with the backend-selected next task. The adaptation reason explains why its difficulty was chosen.
6. After each five-task milestone, answer the satisfaction, perceived-difficulty, and confidence survey.
7. Continue until the curriculum-complete state is reached.

If a page refresh occurs, the stored session token can be used to restore the current task, aggregates, completion count, and any unanswered survey milestone.

Instructor/developer workflow

- Inspect engine inputs and active rules in the submission response's `engineTrace`.
- Use the training-data export for tables and offline analysis.
- Reproduce the synthetic model by running the seed, train, and evaluate management commands documented in the repository README.
- Use Swagger to verify frontend payloads. The browser must never send `isCorrect`, `taskMetricWeight`, or private solution data during the session workflow.

Verification and evaluation

Automated backend verification includes:

- ANFIS strong/weak evidence and output-bound tests;
- deterministic synthetic-row generation;
- training-artifact creation and holdout metadata;
- Mamdani behavioral ordering and centroid method tests;
- answer-safe public task contracts;
- invalid-query 400 responses;
- rejection of client correctness and stale tasks;
- null correctness for non-graded code tasks;
- non-repeating adaptation and module progression; and
- persistent, non-duplicated survey milestones.

The verified suite contains 13 tests. Django system checks, migration-drift checks, and OpenAPI schema validation also pass.

Limitations and future improvements

1. Synthetic records demonstrate the model pipeline but do not replace real learner data. With consent and appropriate anonymization, later work should retrain and recalibrate membership parameters from actual sessions.
2. ANFIS premise membership parameters are fixed for interpretability; only consequent parameters are trained. A larger study could compare this transparent variant with hybrid premise/consequent learning.
3. The curriculum is intentionally small. More tasks per concept would improve adaptation choice and enable concept-level mastery estimates.
4. Session mastery is currently aggregate rather than per concept/module. A longer

deployment should maintain separate knowledge components.

5. Anonymous tokens and the telemetry export are suitable for a local assignment demonstration. A deployed multi-user system would require authentication, role-based export access, retention rules, and a production database.
6. The user study remains future work. The existing micro-survey and export mechanisms are ready to support student and educator evaluation without inventing results.

Conclusion

FuzzyTutor provides an operational, explainable adaptive-learning backend rather than a static demonstration formula. It explicitly defines fuzzy variables, membership functions, rules, aggregation, and defuzzification; trains and evaluates a versioned ANFIS consequent model on held-out synthetic data; applies a genuine Mamdani centroid calculation; persists learner behavior and feedback; and converts model outputs into safe, non-repeating curriculum adaptation. The result is well matched to the scope and difficulty of the joint postgraduate assignment while remaining transparent about the absence of real learner data and production deployment controls.